



Playwright Interview Preparation

Interview Questions

1. How do you get all footer links and click the 3rd?	3
2. How do you get all menu links and print them to the console?	4
3. What are the different types of locators in Playwright?	5
4. What is the priority of locators?	6
5. What happens if a locator matches two elements, and how do you fix it?	6
6. How do you handle a list of elements (locator.all())?	7
7. How do you find a button inside a specific row using .filter()?	7
8. Why do we use async/await in Playwright?	8
9 . How do you run tests in parallel / achieve parallel testing?	8
10. How do you run tests serially?	9
11. How do you run a single test?	9
12. How do you skip a test?	9
13 What command is used to run tests?	10
14. How do you perform tagging in Playwright?	10
15. What is the automation framework structure for Playwright?	11
16. What is a fixture? Do you create your own, and how do you call them?	12
17. What are the types of hooks in Playwright?	13
18. How do you create functions in Playwright?	13
19. How do you manage test data according to environments?	14

1. How do you get all footer links and click the 3rd?

```
import { test, expect } from '@playwright/test';

test('footer links, click 3rd', async ({ page }) => {
  await page.goto('https://www.wikipedia.org/');

  const footerLinks = page.locator('footer a');

  await expect(footerLinks).not.toHaveCount(0); // assert, no sleep
  console.log('count:', await footerLinks.count());

  await footerLinks.nth(2).click(); // nth() is 0-indexed
});
```

2. How do you get all menu links and print them to the console?

```
test('print menu links', async ({ page }) => {  
  await page.goto('https://en.wikipedia.org/');  
  const links = page.locator('#vector-main-menu a, #p-navigation a');  
  
  for (const link of await links.all()) {  
    const text = (await link.innerText()).trim();  
    const href = await link.getAttribute('href');  
    console.log(`${text} -> ${href}`);  
  }  
});
```

Remember:

- ✓ Locators are lazy and retry. ElementHandles (\$, \$\$) are snapshots and go stale.
- ✓ `.all()` and `.count()` do NOT retry. Assertions like `toHaveCount` DO.

3. What are the different types of locators in Playwright?

Two families. User-facing locators resolve against the accessibility tree or visible text, engine-based locators resolve against raw markup.

Family	Locators
User-facing (preferred)	<code>getByRole,</code> <code>getByLabel,</code> <code>getByPlaceholder,</code> <code>getByText,</code> <code>getByAltText,</code> <code>getByTitle,</code> <code>getByTestId</code>
Engine-based (fallback)	<code>locator('css=...'), locator('xpath=...'),</code> <code>:has(), :has-text(), layout engines (:right-of,</code> <code>:near)</code>

- `getByRole` is important.
- It **does not read** `role="..."` off the DOM, it computes the **implicit ARIA role** (a button element is role button, an anchor with href is role link) and the **accessible name** per the W3C accessible name computation: aria-labelledby, then aria-label, then associated label, then text content, then title.
- That is why it survives CSS and structural refactors.

4. What is the priority of locators?

Rank	Locator	Why
1	<code>getByRole</code>	Matches what a user and assistive tech perceive
2 to 6	<code>getByLabel</code> , <code>getByPlaceholder</code> , <code>getByText</code> , <code>getByAltText</code> , <code>getByTitle</code>	User-visible, resilient to refactors
7	<code>getByTestId</code>	Explicit but invisible to users, no a11y signal
8	CSS / XPath	Binds to implementation detail, breaks on refactor

5. What happens if a locator matches two elements, and how do you fix it?

- It throws a **strict mode violation**, and that is intentional.
- Strict mode is on by default for every locator action and most assertions.
- At action time Playwright resolves the selector and asserts the match set is exactly one (after waiting for at least one).
- Two matches is an error, not a silent guess.
- Fixes, in order of preference:

Fix	Example
Narrow the locator	<code>getByRole('button', { name: 'Save' })</code>
Scope to a parent	<code>section.getByRole('button')</code>
Filter by content	<code>.filter({ hasText: 'Row 3' })</code>
Index (last resort)	<code>.first()</code> , <code>.last()</code> , <code>.nth(i)</code>

6. How do you handle a list of elements (locator.all())?

```
const rows = page.getByRole('row');
await expect(rows).toHaveCount(10);           // polls until 10
await expect(rows).toHaveText([/* ... */]);    // ordered, retries

for (const row of await rows.all()) { /* read-only */ }
```

- The distinction is **snapshot vs polled**. `.all()` and `.count()` are one-shot queries with no retry. `toHaveCount` and `toHaveText` re-run the query on a polling loop until the expectation passes or the timeout hits.
- Never act (click, fill) on `.all()` results in a list that mutates, because those handles can detach.

7. How do you find a button inside a specific row using `.filter()`?

```
const row = page.getByRole('row').filter({ hasText: 'John Doe' });
await row.getByRole('button', { name: 'Edit' }).click();

// filter by a nested locator instead of text:
const activeRow = page.getByRole('row')
  .filter({ has: page.getByText('Active') });
```

- `.filter()` compiles to an internal `has-text` or `has-engine` and **AND-combines** it with the base locator, so `getByRole('row').filter(...)` means row AND contains X.
- The chained `getByRole('button')` then scopes the search to inside the matched row's subtree.
- This binds the action to the correct row by content, not by a brittle index.

Remember:

- ✓ Strict mode is a feature: **two matches = error**, so you never click the wrong element silently.
- ✓ `filter-then-chain` is the robust table pattern. Index is the last resort.

8. Why do we use async/await in Playwright?

- Every Playwright command is an **out-of-process RPC**.
- Your test runs in Node, the browser runs in a separate process, and the two talk over a WebSocket using the Playwright protocol (Chromium via CDP, plus Playwright's patched Firefox and WebKit builds).
- So `page.click()` serializes a command, sends it, and the browser does the work asynchronously and replies.
- It must return a Promise, and `await` waits for that round trip plus the auto-waiting loop before the next line runs.
- Drop the `await` and the Promise floats on the microtask queue, the next line fires immediately, and you get an unhandled rejection plus a race where step two starts before step one finished.
- That is the classic flake source.
- `expect()` is sync, but `await expect(locator).toBeVisible()` is async because the matcher polls.
- Forgetting that `await` is a silent no-op assertion.

9 . How do you run tests in parallel / achieve parallel testing?

- By default Playwright spawns workers, each a separate Node process running its own browser instance, and distributes test files across them.
- Files run in parallel, tests within a file run serially unless you opt in.

```
// playwright.config.ts
export default defineConfig({
  workers: process.env.CI ? '50%' : 4, // ~half CPU cores
  fullyParallel: true,                 // parallel WITHIN files
});
```

- Isolation is per **BrowserContext**, not per browser.
- Each test gets a fresh context (incognito-like: clean cookies, storage, cache), cheap to create, while the heavy browser process is reused inside a worker.
- Worker-scoped fixtures persist across tests in that worker.
- For scale across machines you **shard**: `--shard=1/3, 2/3, 3/3` on three CI nodes, then merge the blob reports. A crashed worker restarts, which means `beforeAll` can run more than once.

10. How do you run tests serially?

```
test.describe.configure({ mode: 'serial' });
```

- Serial mode pins the block to a single worker, runs in declared order, and **cascades**: the first failure skips the rest, and on retry it retries the whole block from the top. workers: 1 makes the entire run serial.
- It defeats isolation and speed, so reserve it for genuinely stateful flows.

11. How do you run a single test?

```
npx playwright test tests/login.spec.ts      # file
npx playwright test login.spec.ts:23         # exact line
npx playwright test -g "user can log in"     # title regex
npx playwright test --project=chromium login.spec.ts
```

test.only() works locally, but set **forbidOnly: true** in CI so a committed .only fails the build instead of silently skipping the suite.

12. How do you skip a test?

```
test.skip('wip', async () => {});
test('mobile', async ({ isMobile }) => {
  test.skip(!isMobile, 'reason');
});
test.fixme('known broken', async () => {});
test.describe.skip('block off', () => {});
```

test.skip(condition) inside the body is evaluated at runtime, so you can gate on fixtures like browser name, project, or env. fixme means broken, do not run, which reads better in reports than a generic skip

13 What command is used to run tests?

```
npx playwright test
```

Daily flags:

- --ui (time-travel UI mode),
- --debug (Inspector),
- --headed,
- --trace on,
- --repeat-each=N for flake hunting,
- --grep,
- --workers,
- --shard.

14. How do you perform tagging in Playwright?

```
test('login', { tag: '@smoke' }, async () => {});  
test('big', { tag: ['@regression', '@e2e'] }, async () => {});  
test.describe('checkout', { tag: '@critical' }, () => {});
```

Run or exclude by tag:

```
npx playwright test --grep @smoke  
npx playwright test --grep-invert @slow  
npx playwright test --grep "@smoke|@critical"
```

Remember:

- ✓ Workers = separate processes with their own browser. Isolation is per BrowserContext.
- ✓ Shard across machines and merge blob reports for scale.
- ✓ Never commit .only: set forbidOnly in CI.

15. What is the automation framework structure for Playwright?

```
tests/      feature-grouped specs
pages/      Page Objects (locators + actions)
fixtures/   custom fixtures (auth, data, API)
utils/      pure helpers
test-data/  env-specific JSON
playwright.config.ts  projects, reporters, retries
```

16. What is a fixture? Do you create your own, and how do you call them?

- A fixture provisions state for a test and tears it down.
- Built-ins include page, context, browser, request, browserName.
- You create your own with test.extend, and call them by destructuring them in the test arguments.

```
export const test = base.extend<
  { loginPage: LoginPage },
  { dbSeed: void }
>({
  loginPage: async ({ page }, use) => {
    const lp = new LoginPage(page);
    await lp.goto();
    await use(lp); // before use() = setup, after = teardown
  },
  dbSeed: [
    async ({}, use) => { await seed(); await use(); await clean(); },
    { scope: 'worker', auto: true }, // once per worker
  ],
});
```

17. What are the types of hooks in Playwright?

Hook	Runs
beforeAll	Once before all tests in the file or describe (per worker)
afterAll	Once after all of them
beforeEach	Before every test
afterEach	After every test, receives testInfo (status, retry)

Under the hood. beforeAll runs once **per worker per describe**, not globally once, and because a failed worker restarts it can run multiple times in a flaky run. For one-time cross-worker setup use a global setup project or globalSetup, not beforeAll.

18. How do you create functions in Playwright?

Plain TypeScript, async whenever they await Playwright. Senior tip: have helpers return Locators rather than awaited values, so callers can chain and the laziness is preserved.

```
const rowFor = (page, name) =>
  page.getByRole('row').filter({ hasText: name });

await rowFor(page, 'Jatin')
  .getByRole('button', { name: 'Edit' }).click();
```

19. How do you manage test data according to environments?

Decouple data from logic and switch by environment variable, with dotenv per env:

```
dotenv.config({ path: `.${env.TEST_ENV ?? 'dev'}` });
```

Or use projects so each environment carries its own baseUrl and storageState:

```
projects: [  
  { name: 'staging', use: { baseUrl: process.env.STAGING_URL } },  
  { name: 'prod', use: { baseUrl: process.env.PROD_URL } },  
]
```

- Data lives in JSON per env, the active set is chosen by TEST_ENV, secrets stay in env vars or a vault and never in the repo.
- You can also expose env values as option fixtures so tests read testData instead of reaching into files.